Software Development Methodologies: Troubleshoot Application

Made by the L2_07 team consisting of:
Oleksandr Haishuk (id - 53626)
Mehmet Çeliksu (id - 55893)
Tanatswa Tabagwisa (id - 59272)
Shannaya Cumaio (id - 64342)
Ataberk Güzeller (id - 56450)

Github repository link: https://github.com/OHaishuk/Software-Development-Methodologies-Project

Contents:

Project Goals (page 3-4)

Requirement Specifications (page 5-6)
Development Risks (page 7-8)
User journey (page 9-10)

Glossary list (page 11)

Final Thoughts (page 12)

Project Goals

The core idea of this project is to create a troubleshooting environment that allows quick feedback between developers and their userbase.

This environment would have:

- **Easier troubleshooting** with easy access for users and testers to report problems.
- **An organized layout** for developers, with tickets sorted by priority and deadline.
- **A flexible structure** that can fit into any application, with configurable categories and departments.

How it would work

The main focus of this project is a system that prioritizes problems and splits them to specific people or departments, based on the flow proposed by Tanatswa Tabagwisa:

1. A user reports a problem.
2. The system creates a ticket and assigns a priority (Low, Medium, High or Critical).
3. The ticket is assigned to a developer or support agent.
4. The system tracks the time remaining to resolve the issue based on its SLA (Service Level Agreement).
5. The developer updates the ticket until the issue is resolved.
6. A dashboard displays open tickets, resolved issues, overdue cases and overall performance.

"Definition of done"

The deliverables of this project would be:

- ✓ Finished documentation of an application that explains most, if not all the minute details of how it works
- ✓ A prototype of the application (either a UI prototype made in Figma or a Python application?)
- ✓ A brief report explaining the contribution of everyone involved in this project
- ✓ GitHub repository containing both documentations and the prototype itself

Out of scope items

The deliverables of this project would not be:

- × Integration with the third-party applications
- × Login system
- × A server/database storing all created tickets, messages or accounts
- × Our own website domain

Overall, this application would allow for easy problem reports, saving the testing hassle for the developers, and makes it easier for the users to deal with their frustrations regarding the product.

Requirement Specifications

In this section, the functionality explained in a more specific and grounded way.

Ticket-specific features

Feature	Description
Ticket system	Allows the user to create a ticket with a title, category and description of the problem. Each ticket gets a unique ID.
Attachments	Allows the user to attach a file (e.g. screenshot or log file) to a ticket.
Priority	The system sets Low, Medium, High or Critical from the impact (only me / my team / everyone) and urgency (can wait / slows work / work stopped) the user selects.
Status tracker	Tracks the priority, the time left to solve the problem (SLA: Critical 2 h, High 5 h, Medium 1 day, Low 3 days) and the status: New → In progress → Resolved.
Assignment	Each ticket is assigned to a developer or department, by category or manually by the team lead.
Updates and history	Developers add notes to a ticket; every status change is saved with time and author.

General features

Feature	Description
Ticket list	Shows existing tickets with their status, priority and time left, sorted by closest deadline.
My tickets	Shows the developer only the tickets assigned to them, or the user the tickets that they have made
Dashboard	Shows open, resolved and overdue tickets and the share solved within SLA.
Search and filter	Filters the list by status, priority, category or assignee.

Development risks

In this section, outside factors are mentioned that could potentially affect the schedule or the outcomes of the project:

Risk Table

Risk name	Risk chance	Risk impact	Description
Member Availability	High	Medium	Exams, other courses or illness can leave members with little time in some weeks. Tasks may then be delayed or done in a rush. We limit this with weekly check-ins and by reassigning blocked tasks early.
Scope Creep	Medium	High	New ideas such as notifications or integrations could be added during the semester. This would make it hard to finish the core features on time. We keep an out-of-scope list and only add work through the prioritized backlog.
Limited Access to Test Users	Medium	Medium	We may not find enough real users or testers to try the prototype. Without feedback, our assumptions about what users need cannot be checked. We plan to use classmates and friends as test users.
Technical Difficulties With the Prototype	Medium	Low	Some features, such as the SLA countdown, may be harder to build than expected with our experience. Because the prototype is optional in our Definition of Done, this mainly affects the demo, not the documentation.
Loss of Work or Repository Problems	Low	High	Files could be lost, overwritten or caught in merge conflicts. This could cost days of work close to a deadline. We keep everything

			in the shared GitHub repository and merge only through reviewed pull requests.

User journey

In this section, there are three user scenario examples, briefly describing their individual traits and journey in a format described by the user themselves.

Hannah (Application user)

"Hello, my name's Hannah, and I am 21 years old. I like when my day goes as planned, to which I aid myself with a daily calendar app."

Frustrations: "However, that app is quite buggy, having some problems like sometimes not saving the text I write, or some button looking weird, and I have no idea how to report these aside from maybe sending emails, but that will take a long time."

User Journey: "However, I found out in the calendar app that they suggest installing another one where I can report issues... Not sure why I'd need an extra app, but okay...

And when I open it, I see that I can just pick the app, and see all the existing reports.

I see that some of the bugs had been reported, so I decide to report some of the more obscure stuff.

The only thing left is to wait until they drop an update, in which I am confident now that it will happen sooner or later."

Daniel (Software Tester)

"Hello, my name is Daniel, and I am 26 years old. I work as a software tester in a team that develops a messaging application. My job is to test new features and make sure the application works correctly before updates are released."

Frustrations: "Sometimes, I find several bugs during testing, but reporting and tracking them is difficult. Some reports get lost in long conversations, and I cannot always see whether a developer is working on the issue or if it has been fixed."

User Journey: "During a team meeting, my colleagues and I discussed the difficulties faced when tracking bugs and organizing reports. One of our developers suggested using this troubleshooting system to make the process more organized and efficient.

Our team started using a troubleshooting system to organize bug reports. When I find a problem, I open the application and create a new ticket with a title, category, and description. I attach a screenshot and select the impact and urgency of the problem. The system assigns a priority to the ticket and sends it to the appropriate developer or department.

Later, I checked the ticket list to see its current status and how much time has remained before the deadline. I can see when the developer starts working on the issue and when it's resolved. This makes it easier for me to follow the progress of reported bugs and verify the fixes during testing."

Collin (Application Developer)

"Hello, the name's Collin, and I am workin' in a team on a chatting app as the oldest and experienced developer in the group, being 39."

Frustrations: "Despite my experience, I never use the app we are developing. Even if we have a tester in our team, it's just not enough. We have bunch of user reports in our system, but it's extremely unorganized."

User journey: "One of our team members sent a link to an app that supposedly would help us organize things like bug reports better.

While skeptical, looking into its description revealed that it's... Somewhat easy to integrate."

We decided to give it a shot, and... There were some app-breaking glitches revealed by the users that I was never even aware of... At least I know which code to fix or rewrite."

Glossary list

Name	Description
User	A figure that uses the application, with varying purposes and roles (being a tester for example)
Developer	A person that works on an application (writing code for example)
Tester	A person that helps the developers to find issues within the application that they are developing
Application/app/program/software	A program that is used by the users as a tool to achieve certain goals (using a camera program as a camera for example)
Code	Text that makes the application work
Bug	An error or defect in a software application that causes it to behave incorrectly.
Feature	A task or a novelty that the application can do
Troubleshooting	Reporting/telling the issues (of a program usually)
Ticket	A record of a reported problem that needs to be investigated or resolved.
Priority	The level of importance assigned to a reported issue (Low, Medium, High, or Critical)
SLA (Service Level Agreement)	The target time within which a reported issue should be resolved.
Status	The current stage of a ticket (New, In Progress, or Resolved)

Final thoughts

While unfinished, this project is an amazing opportunity to learn how to work in a team, working on a singular, yet shared goal.